

Linux Filesystem

Exercise 1 – Identify Your Linux Filesystem

Explore the storage configuration of your Linux system.

Determine:

1. The storage devices available.

Storage devices available: sda (80.1G disk) and sr0 (a 1024M optical/CD-ROM drive, TYPE=rom).

2. The partitions available on each device.

Partitions on sda: sda1 (80.1g)

3. The filesystem type used by each partition.

Filesystem type per partition: sda1 → ext4

4. The device/partition on which '/' is located..

Device / is on: /dev/sda1.

5. The filesystem type used for '/'.

Filesystem type for /: ext4

6. The total, used, and available storage space.

Total/used/available space for /: from df -h → 79G total, 17G used, 58G available, 23% used.

```
(kali@kali)-[~]
└─$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda           8:0    0 80.1G 0 disk
└─sda1        8:1    0 80.1G 0 part /
sr0          11:0    1 1024M 1 rom

(kali@kali)-[~]
└─$ lsblk -f
NAME FSTYPE FSVER LABEL UUID                               FSAVAIL FSUSE% M
OUNTPOINTS
sda
└─sda1
    ext4   1.0   root  d8f71766-9813-406b-90f2-fd6cd673b79e  57.6G   21% /
sr0

(kali@kali)-[~]
└─$ df -T
Filesystem      Type      1K-blocks    Used Available Use% Mounted on
udev            devtmpfs   998640         0   998640    0% /dev
tmpfs           tmpfs     201640         980   200660    1% /run
/dev/sda1       ext4     82083148 17433072 60434528   23% /
tmpfs           tmpfs     1008184         4   1008180    1% /dev/shm
none            tmpfs       1024         0      1024    0% /run/credentials/sy
stemd-journald.service
tmpfs           tmpfs     1008184         8   1008176    1% /tmp
none            tmpfs       1024         0      1024    0% /run/credentials/ge
tty@tty1.service
tmpfs           tmpfs     201636        108   201528    1% /run/user/1000

(kali@kali)-[~]
└─$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            888M   0 888M   0% /dev
tmpfs           197M  980K 196M   1% /run
/dev/sda1       79G   17G  58G   23% /
tmpfs           985M  4.0K 985M   1% /dev/shm
none            1.0M   0 1.0M   0% /run/credentials/systemd-journald.servi
ce
tmpfs           985M  20K 985M   1% /tmp
none            1.0M   0 1.0M   0% /run/credentials/getty@tty1.service
tmpfs           197M 108K 197M   1% /run/user/1000
```

Record your findings in a table.

Device/Partition	Filesystem Type	Size	Mount Point
/dev/sda1	ext4	80.1G	/
/dev/sr0	-	1024M	— (optical, drive)

Exercise 2 – Disk → Partition → Filesystem → Mount Point

Using the information obtained in Exercise 1, draw the storage organization of your own Linux system. Explain in 2–3 sentences the relationship between these four components.

sda (80.1G disk)

|

+-- sda1 (80.1G partition)

|

+-- ext4 (filesystem)

|

+-- / (mount point)

sr0 (1024M optical/CD-ROM device)

|

+-- (no partition/filesystem shown)

The physical disk sda (80.1G) contains a single partition, sda1 (80.1G), which is formatted with the ext4 filesystem. The ext4 filesystem is mounted at /, making its files and directories accessible through the Linux directory tree, while sr0 is an optical/CD-ROM device with no partition or filesystem currently shown.

Exercise 3 – Explore the Root Filesystem

Start from your current location.

1. Determine your current working directory.
2. Navigate to the root directory.
3. Display the contents of the root directory.
4. Identify all directories directly available under `/`.
5. Determine whether hidden entries are present.

```
(kali㉿kali)-[~]
└─$ pwd
/home/kali

(kali㉿kali)-[~]
└─$ cd /

(kali㉿kali)-[/]
└─$ pwd
/

(kali㉿kali)-[/]
└─$ ls
bin  dev  home  initrd.img.old  lib32  lost+found  mnt  proc  run  srv  sys  usr  vmlinuz
boot  etc  initrd.img  lib  lib64  media  opt  root  sbin  swap  tmp  var  vmlinuz.old

(kali㉿kali)-[/]
└─$ ls -a
.  bin  dev  home  initrd.img.old  lib32  lost+found  mnt  proc  run  srv  sys  usr  vmlinuz
.. boot  etc  initrd.img  lib  lib64  media  opt  root  sbin  swap  tmp  var  vmlinuz.old

(kali㉿kali)-[/]
└─$
```

Exercise 4 – Linux Filesystem Hierarchy Investigation

For each location:

1. Examine its contents.
2. Identify at least three entries, where available.
3. Determine the likely purpose of the directory.

Directory	Example Contents	Purpose
/home	kali	Holds personal directories for each regular user account; each user's folder is locked to that user only
/root	-	Home directory for the root (superuser) account, kept separate from /home and restricted (mode 700)
/etc	-	System-wide configuration files for the OS and all installed applications/services
/var	-	Variable data that changes during operation — logs, caches, mail spool, web files
/var/log	-	Stores system, boot, login, and application logs used for troubleshooting and auditing
/tmp	-	World-writable temporary storage for session/runtime files; sticky bit set; cleared on reboot
/usr	-	Holds installed user programs, libraries, and shared documentation/data
/boot	initrd.img, initrd.img.old, vmlinuz, vmlinuz.old	Contains the kernel image, initial ramdisk, and bootloader files needed to start the system
/dev	sda, sda1, sr0	Device files representing hardware and virtual devices, accessed like normal files
/proc	-	Virtual, kernel-generated filesystem exposing live process and system information; nothing stored on disk

Exercise 5 – Find Your Home Directory

Without assuming its location:

1. Determine your username.
2. Determine your home directory.
3. Navigate to your home directory.
4. Display its absolute path.
5. List its normal contents.
6. List all contents, including hidden files.

```

(kali@kali)-[/]
$ echo $HOME
/home/kali

(kali@kali)-[/]
$ cd ~

(kali@kali)-[~]
$ pwd
/home/kali

(kali@kali)-[~]
$ ls
Desktop  Downloads  extended  globbing_practice  LinuxOS_2026  Music
Documents example filters linux_lab

(kali@kali)-[~]
$ ls -la
total 224
drwxr-xr-x 25 kali kali 4096 Sep  5 11:03 .
drwxr-xr-x  3 root root 4096 Jun 16 10:30 ..
-rw-r--r--  1 kali kali 1682 Aug 15 14:23 .bash_history
-rw-r--r--  1 kali kali 220 Jun 16 10:30 .bash_logout
-rw-r--r--  1 kali kali 5578 Jun 16 10:30 .bashrc
-rw-r--r--  1 kali kali 3526 Jun 16 10:30 .bashrc.original
drwxrwxr-x 11 kali kali 4096 Aug 11 07:06 .cache
drwxr-xr-x 16 kali kali 4096 Aug 15 12:45 .config
drwxr-xr-x  2 kali kali 4096 Aug  8 11:43 Desktop
-rw-r--r--  1 kali kali 35 Aug  8 11:43 .dmrc
drwxr-xr-x  2 kali kali 4096 Sep  5 11:30 Documents
drwxr-xr-x  2 kali kali 4096 Aug 15 11:03 Downloads
drwxrwxr-x  2 kali kali 4096 Aug 10 12:30 example
drwxrwxr-x  2 kali kali 4096 Aug 15 14:30 extended
-rw-r--r--  1 kali kali 11759 Jun 16 10:30 .face
lrwxrwxrwx  1 kali root  5 Jun 16 10:30 .face.icon -> .face
drwxrwxr-x  2 kali kali 4096 Aug 15 14:33 filters
-rw-r--r--  1 kali kali 62 Aug 10 15:00 .gitconfig
drwxrwxr-x  2 kali kali 4096 Aug 15 14:25 globbing_practice
drwxr-xr-x  3 kali kali 4096 Aug  8 11:43 .gnupg
-rw-r--r--  1 kali kali 0 Aug  8 11:43 .ICEauthority
drwxr-xr-x  3 kali kali 4096 Jun 16 10:30 .java
drwxrwxr-x  3 kali kali 4096 Aug 25 12:55 linux_lab
drwxrwxr-x  6 kali kali 4096 Aug 15 14:51 LinuxOS_2026
drwxr-xr-x  5 kali kali 4096 Aug  8 11:43 .local
drwxrwxr-x  5 kali kali 4096 Aug 11 07:06 .mozilla
drwxr-xr-x  2 kali kali 4096 Aug  8 11:43 Music

```

Exercise 6 – Hidden Files Investigation

Examine your home directory.

Compare:

ls

ls -a

Identify at least 2 hidden files/directories.

- .bash_history — stores your shell command history
- .ssh — hidden directory holding SSH keys and configuration

```

(kali@kali)-[~]
$ cd ~

(kali@kali)-[~]
$ ls
Desktop  Downloads  extended  globbing_practice  LinuxOS_2026  phishing_simulator  Projects  Templates
Documents example filters linux_lab Music Pictures Public Videos

(kali@kali)-[~]
$ ls -a
.
..
.bash_history
.bash_logout
.bashrc
.bashrc.original
.cache
.config
Desktop
.dmrc
Documents
Downloads
example
extended
.face
.face.icon
filters
.gitconfig
globbing_practice
.gnupg
.ICEauthority
.java
linux_lab
LinuxOS_2026
.local
.mozilla
Music
phishing_simulator
Pictures
.profile
Projects
Public
.ssh
.sudo_as_admin_successful
.templates
.vboxclient-clipboard-tty7-control.pid
.vboxclient-clipboard-tty7-service.pid
.vboxclient-draganddrop-tty7-control.pid
.vboxclient-draganddrop-tty7-service.pid
.vboxclient-hostversion-tty7-control.pid
.vboxclient-seamless-tty7-control.pid
.vboxclient-seamless-tty7-service.pid
.vboxclient-vmsvga-session-tty7-control.pid
.vboxclient-vmsvga-session-tty7-service.pid
.xauthority
.xsession-errors
.xsession-errors.old
.zprofile
.zsh_history
.zshrc

```

1. How does Linux identify a hidden file?

Linux treats a file or directory as hidden when its name starts with a dot (.). In your output, entries such as .bash_history, .ssh, .bashrc, and .config are hidden because their names begin with ..

2. Why does a normal ls not display these files?

The normal ls command does not display dot-prefixed files and directories by default. This keeps configuration files, cache files, and other hidden entries from cluttering the normal directory listing. It is a visibility convention, not a security feature.

3. What do . and .. mean?

- . represents the current directory.
- .. represents the parent directory.

Both are visible in your ls -a output.

4. Can a hidden file still be accessed if its name is known?

Yes. A hidden file or directory can still be accessed if its name is known and the user has the required permissions. The dot only

affects whether it is displayed by commands such as normal ls.

5. From a cybersecurity perspective, why should hidden files not automatically be considered safe or malicious?

Hidden files should not automatically be considered safe or malicious because the dot prefix only indicates that the item is hidden from normal directory listings. Your own home directory demonstrates this: .bashrc, .profile, and .cache are ordinary system/user files, while .ssh may contain sensitive configuration or key files. Therefore, a cybersecurity investigator should examine the contents, permissions, ownership, and context of a hidden file rather than assuming that being hidden makes it suspicious or safe.

Exercise 7 – Build a Small Filesystem Hierarchy

Explore: mkdir

Hint: you can create this hierarchy in a single command

Create the following files:

cyber_lab/evidence/network/traffic.txt

cyber_lab/evidence/system/processes.txt

cyber_lab/logs/security.log

cyber_lab/scripts/check.sh

cyber_lab/reports/investigation.txt

Use appropriate Linux commands to create the structure.

Verify that the required hierarchy has been created successfully.

Hint: tree

Explore options

```
(kali@kali):[~]
$ cd ~
(kali@kali):[~]
$ mkdir -p cyber_lab/evidence/network cyber_lab/evidence/system cyber_lab/logs cyber_lab/scripts cyber_lab/report
(kali@kali):[~]
$ touch cyber_lab/evidence/network/traffic.txt \
  cyber_lab/evidence/system/processes.txt \
  cyber_lab/logs/security.log \
  cyber_lab/scripts/check.sh \
  cyber_lab/reports/investigation.txt
(kali@kali):[~]
$ tree cyber_lab
cyber_lab
├── evidence
│   ├── network
│   │   └── traffic.txt
│   └── system
│       └── processes.txt
├── logs
│   └── security.log
├── reports
│   └── investigation.txt
├── scripts
│   └── check.sh
└── report

8 directories, 5 files
(kali@kali):[~]
$ mkdir -p cyber_lab/evidence/network cyber_lab/evidence/system cyber_lab/logs cyber_lab/scripts cyber_lab/report
(kali@kali):[~]
$ touch cyber_lab/evidence/network/traffic.txt \
  cyber_lab/evidence/system/processes.txt \
  cyber_lab/logs/security.log \
  cyber_lab/scripts/check.sh \
  cyber_lab/reports/investigation.txt
```

Exercise 8 – Absolute and Relative Paths

Assume your current directory is:

~/cyber_lab

Perform the following:

1. Navigate to `evidence/network` using a relative path.
2. Navigate to `/etc` using an absolute path.
3. Return to your home directory using the shortest possible command.
4. Navigate to `cyber\_lab/reports`.
5. Move one directory upward.
6. Display the current directory after every operation.

```
(kali@kali)-[~]
$ cd ~/cyber_lab
(kali@kali)-[~/cyber_lab]
$ cd evidence/network
(kali@kali)-[~/cyber_lab/evidence/network]
$ pwd
/home/kali/cyber_lab/evidence/network
(kali@kali)-[~/cyber_lab/evidence/network]
$ cd /etc
(kali@kali)-[/etc]
$ pwd
/etc
(kali@kali)-[/etc]
$ cd ~
(kali@kali)-[~]
$ pwd
/home/kali
(kali@kali)-[~]
$ cd
(kali@kali)-[~]
$ cd cyber_lab/reports
(kali@kali)-[~/cyber_lab/reports]
$ pwd
/home/kali/cyber_lab/reports
(kali@kali)-[~/cyber_lab/reports]
$ cd ..
(kali@kali)-[~/cyber_lab]
$ pwd
/home/kali/cyber_lab
```

* What is an absolute path?

An absolute path starts from the filesystem root / and completely specifies the location of a file or directory, regardless of the current working directory.

Eg- /home/kali/cyber_lab/evidence/network

* What is a relative path?

A relative path specifies a location relative to the current working directory. It does not start with /.
cyber_lab.

* What is the difference between '/' and '~'?

/ is the root directory of the entire Linux filesystem and is at the top of the directory hierarchy.

~ is a shortcut representing the current user's home directory.

Exercise 9 – Identify File Types

Investigate the following:

/etc/passwd

/etc

/bin/bash

/dev/null

/proc

Also investigate the files created in Exercise 7.

file <filename>

ls -l <filename>

Classify each object.

```

kali@kali:~/cyber_lab$ file /etc/passwd
/etc/passwd: ASCII text

kali@kali:~/cyber_lab$ file /etc
/etc: directory

kali@kali:~/cyber_lab$ file /bin/bash
/bin/bash: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=e1983fd58f88f72cd9b892c28692a8eef51d8da1, for GNU/Linux 3.2.0, stripped

kali@kali:~/cyber_lab$ file /dev/null
/dev/null: character special (1/3)

kali@kali:~/cyber_lab$ file /proc
/proc: directory

kali@kali:~/cyber_lab$ ls -l /etc/passwd
-rw-r--r-- 1 root root 3256 Jun 16 10:30 /etc/passwd

kali@kali:~/cyber_lab$ ls -l /etc
drwxr-xr-x 185 root root 12288 Sep  5 11:02 /etc

kali@kali:~/cyber_lab$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1384752 May  3 15:28 /bin/bash

kali@kali:~/cyber_lab$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Sep  5 11:02 /dev/null

kali@kali:~/cyber_lab$ ls -l /proc
dr-xr-xr-x 250 root root 0 Sep  5 11:02 /proc

kali@kali:~/cyber_lab$ cd ~/cyber_lab

kali@kali:~/cyber_lab$

```

Object	Type
/etc/passwd	Regular text file (ASCII text — user account database)
/etc	Directory
/bin/bash	Regular file — ELF executable binary (or symlink to /usr/bin/bash on modern Kali)
/dev/null	Character special device file
/proc	Directory — root of the virtual/pseudo procfs filesystem
evidence/network/traffic.txt	Regular (empty) file
evidence/system/processes.txt	Regular (empty) file
logs/security.log	Regular (empty) file
scripts/check.sh	Regular (empty) file
reports/investigation.txt	Regular (empty) file

Exercise 10 – Examine File Metadata

Use:

~/cyber_lab/reports/investigation.txt

Add a small amount of text to the file.

Examine it using:

ls -l

stat investigation.txt

Identify:

- * Filename
- * File size
- * File type
- * Owner
- * Group
- * Inode number
- * Access time
- * Modification time
- * Change time

```
(kali㉿kali)-[~/cyber_lab]
$ cd ~/cyber_lab/reports
(kali㉿kali)-[~/cyber_lab/reports]
$ echo "Security investigation report" > investigation.txt
(kali㉿kali)-[~/cyber_lab/reports]
$ ls -l investigation.txt
-rw-rw-r-- 1 kali kali 30 Sep  5 12:38 investigation.txt
(kali㉿kali)-[~/cyber_lab/reports]
$ stat investigation.txt
File: investigation.txt
Size: 30          Blocks: 8          IO Block: 4096   regular file
Device: 8,1      Inode: 1060469   Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/   kali)   Gid: ( 1000/   kali)
Access: 2026-09-05 12:17:21.914737911 -0400
Modify: 2026-09-05 12:38:06.234577905 -0400
Change: 2026-09-05 12:38:06.234577905 -0400
 Birth: 2026-09-05 12:16:41.357115584 -0400
```

Filename: investigation.txt

File size: 30 bytes

File type: regular file

Owner: Kali

Group: Kali

Inode number: 1060469

Access time (atime): 2026-09-05 12:17:21.914737911 -0400

Modification time (mtime):

2026-09-05 12:38:06.234577905 -0400

Change time (ctime): 2026-09-05 12:38:06.234577905 -0400

What is metadata?

Data describing a file — its size, owner, group, permissions, timestamps, and inode — rather than the file's actual content.

Is metadata the same as file content?

No. You can change metadata (e.g. `chmod`, `chown`, `rename`) without touching content, and you can edit content without directly touching most metadata (though `mtime`/`ctime` still update automatically).

Which timestamp changes when file contents are modified?

`mtime` (modification time) changes directly. `ctime` also updates as a side effect (since the inode's size/`mtime` metadata changed), but `atime` only changes on read access, not on write.

Exercise 11 – Explore `/etc`

Investigate:

`/etc`

...

Locate:

`passwd`

`group`

`hostname`

hosts

Note: Without modifying any system files, examine their contents where permitted.

```

rcbqnwb:x:a88:a88:rcbqnwb:\u006172f6u6:\n21\3p7u\u00617u
27iou82mwu:x:j00:0223f::\u00617p7\27iou82mwu:\n21\3p7u\u00617u
f22:x:a80:a80:f22 n261 f01 f6w5:\:\n21\3p7u\u00617u
w6229d6rnp2:x:a88:a88:2\u00617w w6229d6 bnp2:\u006172f6u6:\n21\3p7u\u00617u
2\u00617wq-f7w62\u00617u:x:a80:a80:2\u00617wq j7w6 2\u00617u\u00617u\u00617u:\n21\3p7u\u00617u
-q\u00617cbq:x:a80:a80:DHCB CJ76u6 p96wou:\u00617p7\u00617u\u00617u\u00617u\u00617u
2\u00617wq-u67mou7k:x:0\u00617:0\u00617:2\u00617wq w67mou7k w9u9d6w6u6:\:\n21\3p7u\u00617u
uorouq\u0061:x:0223f:0223f:uorouq\u0061:\u006172f6u6:\n21\3p7u\u00617u
-9b7:x:f5:0223f::\u006172f6u6:\n21\3p7u\u00617u
77c:x:30:30:77cq:\u0061777cq:\n21\3p7u\u00617u
77f7:x:30:30:w9777u6 77f7 w9u9d61:\u0061777f7:\n21\3p7u\u00617u
p9c7nb:x:3f:3f:p9c7nb:\u00617p9c7nb2:\n21\3p7u\u00617u
mm-\u00617f9:x:33:33:mm-\u00617f9:\u00617mm:\n21\3p7u\u00617u
b10x\u0061:x:j3:j3:b10x\u0061:\u00617u:\n21\3p7u\u00617u
nncb:x:j0:j0:nncb:\u00617\u00617b007\u00617u:\n21\3p7u\u00617u
u6m2:x:a:0:u6m2:\u00617\u00617b007\u00617u:\n21\3p7u\u00617u
w977:x:8:8:w977:\u00617w977:\n21\3p7u\u00617u
fb:x:\u00617:fb:\u00617\u00617b007\u00617u:\n21\3p7u\u00617u
w9u:\x:e:j5:w9u:\u00617\u00617c9c\u00617w9u:\n21\3p7u\u00617u
d9w62:x:j:00:d9w62:\n21\d9w62:\n21\3p7u\u00617u
2\u00617u:x:f:0223f:2\u00617u:\u00617u\u00617u
2\u00617u:x:j:3:2\u00617u:\u00617u:\n21\3p7u\u00617u
p7u:x:j5:j5:p7u:\u00617u:\n21\3p7u\u00617u
q96wou:x:j:j:q96wou:\n21\3p7u:\n21\3p7u\u00617u
\u006100f:x:j0:j0:\u006100f:\u006100f:\n21\3p7u\u00617u
c9f \u00617c\u00617u02f2
c9f \u00617c\u00617u02f9w6
c9f \u00617c\u00617u0nb
c9f \u00617c\u00617u0222mq
(k977@k977)-[-\u00617p777p7\u00617b01f2]

```

What kind of information is primarily stored under /etc?

Host-wide, static configuration files controlling the OS and every installed application/service — no user data or binaries.

What does `/etc/passwd` contain?

One line per user account: username, placeholder password field (x, since real hashes live in `/etc/shadow`), UID, GID, GECOS comment, home directory, login shell.

What does `/etc/hostname` contain?

The system's configured hostname (e.g. `kali`), a single line.

Why is `/etc` security-sensitive?

It defines user accounts, authentication rules (`/etc/shadow`, `/etc/sudoers`), network configuration, and service behavior. Unauthorized changes here can create backdoor accounts, escalate privileges, redirect network traffic (`/etc/hosts`), or weaken authentication — as your own `ls -la /etc` output showed, most of it is root-owned and root-writable only.

Exercise 12 – Explore Linux Log Storage

Investigate:

```
/var/log
```

```

└─$ ls -la /var/log
ls -lt /var/log | head          # sorted by most recently modified
sudo cat /var/log/auth.log 2>/dev/null | tail -20
cat /var/log/dpkg.log | tail -20
total 1952
drwxr-xr-x  22 root          root          4096 Aug 31 23:20 .
drwxr-xr-x  12 root          root          4096 Aug 11 10:14 ..
-rw-r--r--   1 root          root        86693 Aug 11 10:12 alternative
drwxr-xr-x   2 root          adm          4096 Aug 22 18:20 apache2
drwxr-xr-x   2 root          root          4096 Aug 20 20:56 apt
-rw-r--r--   1 root          root           0 Aug 31 23:20 boot.log

```

Identify

- 1. Available log files
 - alternatives — file, 86,693 bytes
 - boot.log — file
 - apt — this is a directory, not a file

- ## 2. Available log directories

apache2/

apt/

3. Recently modified entries

The `ls -lt /var/log | head` output is sorted by most recently modified. From the visible entries:

```
boot.log — Aug 31 23:20
. — Aug 31 23:14
.. — Aug 31 23:14
alternatives — Aug 11 10:12
apache2/ — Aug 20 20:56
apt/ — Aug 20 20:56
```

4. Entries you can read

`boot.log` appears readable (`-rw-r----`)

`alternatives` appears readable (`-rw-r--r--`)

The `apache2` and `apt` directories are accessible/listable based on their permissions (`drwxr-xr-x`).

5. Entries you cannot read as a normal user

The screenshot shows most entries owned by `root:root`. For files with permissions such as `-rw-r----`, a normal user who is not in the owning group cannot read them.

Answers

1. Why does Linux maintain logs?

Linux maintains logs to record system, application, authentication, and security-related events. They help administrators understand what happened on the system.

2. Why are logs useful during incident investigation?

Logs provide evidence about events such as logins, failed authentication attempts, service activity, errors, and system changes. Investigators can use timestamps and event details to reconstruct what happened.

3. Why might access to some logs be restricted?

Some logs contain sensitive information, such as authentication details, user activity, or system-security information. Restricting access prevents unauthorized users from viewing or modifying this information and helps preserve the integrity of the logs.

Exercise 13 – Investigate `/tmp`

Explore `/tmp` without modifying files belonging to other users.

Determine:

1. Whether files/directories currently exist.

Yes. `/tmp` contains several files and directories used for temporary files, sockets, and application/session data.

2. Whether hidden files exist.

Yes. Hidden entries are present, such as `.X11-unix`, `.ICE-unix`, and other dot-prefixed entries.

3. Who owns some of the entries.

The entries may be owned by `root` or by the current user, depending on which process created them.

4. Which entries were modified recently.

The command `ls -lt /tmp` displays entries in order of modification time, with the most recently modified entries appearing first.

```

(kali@kali)-[~]
$ ls -la /tmp
total 12
drwxrwxrwt 12 root root 320 Sep  5 12:54 .
drwxr-xr-x 18 root root 4096 Aug 10 14:45 ..
-rw-r--r-- 1 kali kali  0 Sep  5 11:03 config-err-ay8cSs
drwxrwxrwt 2 root root  40 Sep  5 11:02 .font-unix
drwxrwxrwt 2 root root  60 Sep  5 11:03 .ICE-unix
sw-rw-rw-rw- 1 root root  0 Sep  5 11:02 .iprt-localipc-DRMipcServer
drwxrwxrwt 3 root root  60 Sep  5 11:03 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-colord.service-E6aREx
drwxrwxrwt 3 root root  60 Sep  5 11:02 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-ModemManager.service-yMbV58
drwxrwxrwt 3 root root  60 Sep  5 11:05 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-pcsd.service-umYYI3
drwxrwxrwt 3 root root  60 Sep  5 11:02 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-polkit.service-IMRmoi
drwxrwxrwt 3 root root  60 Sep  5 11:02 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-systemd-logind.service-yV
drwxrwxrwt 3 root root  60 Sep  5 11:03 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-upower.service-TERqnm
-r--r--r-- 1 root root 11 Sep  5 11:02 .X0-lock
drwxrwxrwt 2 root root  60 Sep  5 11:02 .X11-unix
-rw-r--r-- 1 kali kali 394 Sep  5 11:03 .xfsm-ICE-EY85U3
drwxrwxrwt 2 root root  40 Sep  5 11:02 .XIM-unix

(kali@kali)-[~]
$ ls -lt /tmp
total 0
drwxrwxrwt 3 root root 60 Sep  5 11:05 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-pcsd.service-umYYI3
drwxrwxrwt 3 root root 60 Sep  5 11:03 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-colord.service-E6aREx
drwxrwxrwt 3 root root 60 Sep  5 11:03 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-upower.service-TERqnm
-rw-r--r-- 1 kali kali  0 Sep  5 11:03 config-err-ay8cSs
drwxrwxrwt 3 root root 60 Sep  5 11:02 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-ModemManager.service-yMbV58
drwxrwxrwt 3 root root 60 Sep  5 11:02 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-systemd-logind.service-yVtmK
drwxrwxrwt 3 root root 60 Sep  5 11:02 systemd-private-64fc52eafcad4a35b27e9b6dd6339fda-polkit.service-IMRmoi

```

Why can temporary directories be relevant during a cybersecurity investigation?

/tmp is relevant because it is a temporary, commonly writable location where programs may create temporary files. During a security investigation, suspicious scripts, downloaded files, extracted archives, or malware-related artifacts may be found there, so investigators should examine it carefully.

Exercise 14 – Explore the `/proc` Filesystem

Investigate:

/proc

...

Examine:

cat /proc/cpuinfo

cat /proc/meminfo

...

Then identify your current shell PID:

echo \$\$

...

Check whether a corresponding directory exists under `/proc`.

Answer:

1. What information does `/proc/cpuinfo` provide?

/proc/cpuinfo — shows details about the CPU(s): model, speed, cache size, and supported features. Since I'm in a VM, it only shows 1 core and flags it as running under a hypervisor.

```

(kali@kali)-[~]
$ cat /proc/cpuinfo
processor       : 0
vendor_id     : GenuineIntel
cpu family    : 6
model         : 140
model name    : 11th Gen Intel(R) Core(TM) i3-1115G4 @ 3.00GHz
stepping      : 1
microcode    : 0xffffffff
cpu MHz       : 2995.198
cache size    : 6144 KB
physical id   : 0
siblings      : 2
core id       : 0
cpu cores     : 2
apicid        : 0
initial apicid : 0
fpu           : yes
fpu_exception : yes
cpuid level   : 22

```

```

(kali@kali)-[~]
$ echo $$
2076

(kali@kali)-[~]
$ ls /proc/$$
arch_status  cgroup      coredump_filter  exe          io          ksm_merging_pages  loginuid  mountinfo  ns
attr         clear_refs   cpu_resctrl_groups  fd           ksm_stat    map_files  mounts     numa_maps
autogroup    cmdline     cwd              fdinfo       ksm_stat    maps       mountstats oom_adj
auxv         comm        environ          gid_map      limits      mem        net        oom_score

(kali@kali)-[~]
$ cat /proc/meminfo
MemTotal:        2016368 kB
MemFree:         214468 kB
MemAvailable:    556316 kB
Buffers:         6824 kB
Cached:          580516 kB
SwapCached:      52480 kB
Active:          967564 kB
Inactive:        596616 kB
Active(anon):    684748 kB
Inactive(anon):  583232 kB
Active(file):    362816 kB
Inactive(file):  93384 kB
Unswappable:     112 kB
Mlocked:         112 kB
SwapTotal:       976556 kB

```

2. What information does `/proc/meminfo` provide?

`/proc/meminfo` — shows live memory stats: how much RAM is total, free, and available, plus swap usage.

3. Why do directories with numbers appear inside `/proc`?

Numbered folders in `/proc` — each one is a running process's PID. My shell (PID 2179) has its own folder with live info like its command line, open files, and memory usage.

4. Is `/proc` an ordinary disk directory?

No — it's built by the kernel on the fly, purely in memory. Nothing here is actually saved to disk, and folders disappear the moment their process ends.

5. Why could `/proc` be useful during security investigation?

it lets you see exactly what's running right now, including a process's real command and open files/connections. That's useful for spotting something suspicious even if it's disguised or hidden from normal tools like `ps`.

Exercise 15 – Explore `/dev`

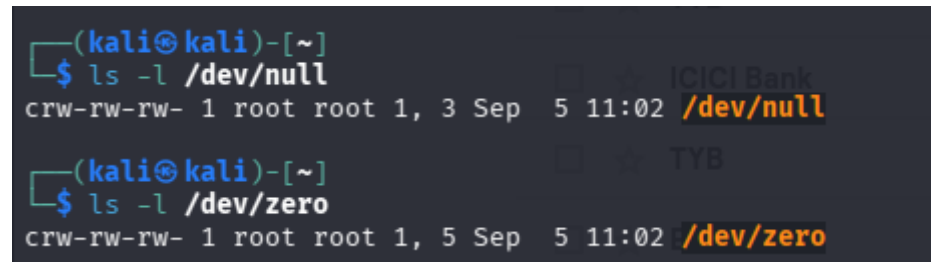
`/dev`

...

Examine:

`ls -l /dev/null`

`ls -l /dev/zero`



```
(kali㉿kali)-[~]
$ ls -l /dev/null
crw-rw-rw- 1 root root 1, 3 Sep  5 11:02 /dev/null

(kali㉿kali)-[~]
$ ls -l /dev/zero
crw-rw-rw- 1 root root 1, 5 Sep  5 11:02 /dev/zero
```

...

What does `/dev` represent?

It's where Linux exposes hardware and virtual devices as files — so programs can interact with things like disks, terminals, and these memory devices through normal file operations.

Are these ordinary user files?

No — the `c` at the start of the permissions marks them as character devices, not regular files. They don't hold data on disk; they're just interfaces to a kernel driver.

File type of `/dev/null`?

Character special device (major 1, minor 3).

Why does Linux expose devices this way?

It follows the "everything is a file" philosophy — instead of needing special device-specific code, any program can just `open`, `read`, or `write` to a device like it's a normal file. `/dev/null` discards anything written to it and returns EOF on read; `/dev/zero` returns endless zero bytes on read.

Exercise 16 – Filesystem Space Investigation

Determine:

Use appropriate combinations of:

`df`

`df -h`

`du`

`du -h`

`du -sh`

```
(kali@kali)-[~]
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            888M    0   888M   0% /dev
tmpfs           197M  992K   196M   1% /run
/dev/sda1       79G   17G   58G  23% /
tmpfs           985M   4.0K   985M   1% /dev/shm
tmpfs           985M  648K   984M   1% /tmp
none            1.0M    0    1.0M   0% /run/credentials/getty@tty1.service
tmpfs           197M  120K   197M   1% /run/user/1000
none            1.0M    0    1.0M   0% /run/credentials/systemd-journald.service

(kali@kali)-[~]
$ du -sh ~
243M /home/kali

(kali@kali)-[~]
$ du -sh ~/cyber_lab
36K /home/kali/cyber_lab
```

Total filesystem capacity: 79G

Used space: 17G

Available space: 58G

Percentage utilization: 23%

Space occupied by home directory: 243M (/home/kali)

Space occupied by cyber_lab: 36K (/home/kali/cyber_lab)

Explain the difference between `df` and `du`.

df vs du

Think of storage like a house:

df tells you how full the whole filesystem is. In this system, the / filesystem has 79G of total space, 17G is used, and 58G is available, with 23% utilization..

du tells you how much space is being used by a particular directory or file. Here, /home/kali uses 243M, while ~/cyber_lab uses only 36K.

Exercise 17 – Cybersecurity Investigation Scenario

Assume that you are investigating a Linux machine after suspicious activity has been reported.

You are initially asked to examine only the filesystem.

Identify which of the following locations you would investigate and explain **why**:

/home

/etc

/var/log

/tmp

/proc

/dev

/root

For each selected location, state what type of security-relevant information you would expect to find.

/home – To look for suspicious files, scripts, downloads, hidden files, and user activity.

/etc – To check for changed settings, new users, sudo changes, or scheduled tasks.

/var/log – To find login activity, errors, and other clues about what happened.

/tmp – Attackers may use it to temporarily store malicious files or scripts.

/proc – Shows what is currently running on the system, which can help find suspicious processes.

/root – If accessible, it can show signs that an attacker gained root/admin access.

Low priority: /dev because it normally contains system device files and is less likely to contain useful evidence.